Perplexity and N-gram Smoothing

Perplexity

- Measurement of how well a probability model predicts a sample.
- In language modeling, it tells us how "surprised" or "perplexed" the model is when it sees new text.
- Lower perplexity = Better model (less surprised by the text)
- Higher perplexity = Worse model (more surprised by the text)

Formula

- Take the probability of the whole sentence, then take the Nth root

$$PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

- actually the inverse probability
- Apply Chain Rule:

$$PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_1 \dots w_{i-1})}}$$

- For bigram models:

$$PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}}$$

PROBLEM

- A bigram language model with the following probabilities:

Bigram	Probability
P(the|<s>)	0.4
P(cat|the)	0.3
P(sat|cat)	0.6
P(on|sat)	0.5
P(mat|on)	0.4
P(</s>|mat)	0.7

- Calculate the perplexity of the sentence: "The cat sat on mat"

Step 1: Sentence representation with start/end tokens

- Sentence: <s> the cat sat on mat </s>
- Number of words (N) = 6 (including end token, excluding start token for calculation)

Step 2: Calculate probability of each bigram

- Sentence: <s> the cat sat on mat </s>
- Number of words (N) = 6 (including end token, excluding start token for calculation)
- P(the|<s>)=0.4
- P(cat|the)=0.3
- P(sat|cat)=0.6
- P(on|sat)=0.5
- P(mat|on)=0.4
- P(</s>|mat)=0.7

Step 3: Calculate sentence probability

- P(sentence)=0.4×0.3×0.6×0.5×0.4×0.7
- P(sentence) = 0.01008

Step 4: Calculate perplexity

- Using formula: PP(W) = $\sqrt[N]{\frac{1}{P(\text{sentence})}}$
- $$PP(W) = \sqrt[6]{\frac{1}{0.01008}}$$

$$PP(W) = \sqrt[6]{99.206}$$

- = 2.15

Final perplexity ≈ 2.15

Linear Interpolation for N-gram Language Models

- When using n-gram models, we face the sparsity problem:
- Higher-order n-grams (trigrams, 4-grams) are more specific but sparse
- Lower-order n-grams (bigrams, unigrams) are less specific but more frequent
- We combine them for better predictions - Linear interpolation provides a simple, effective solution.

Basic Linear Interpolation

- For a trigram model:

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n|w_{n-2}w_{n-1}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n)$$

- Constraints:

$$\lambda_1 + \lambda_2 + \lambda_3 = 1$$

$$0 \leq \lambda_i \leq 1$$

Problem

- Given Corpus Statistics:
- Total words: 10,000
- count("the quick brown"): 50
- count("the quick"): 100
- count("quick cat"): 5
- count("quick"): 500
- count("cat"): 200
- Calculate P(cat|the quick)

Individual Probabilities

1. **Trigram:** $P(\text{cat}|\text{the quick}) = \frac{\text{count}(\text{"the quick cat"})}{\text{count}(\text{"the quick"})} = \frac{0}{100} = 0$
   - Problem! Never seen "the quick cat"
2. **Bigram:** $P(\text{cat}|\text{quick}) = \frac{\text{count}(\text{"quick cat"})}{\text{count}(\text{"quick"})} = \frac{5}{500} = 0.01$
3. **Unigram:** $P(\text{cat}) = \frac{\text{count}(\text{"cat"})}{\text{total words}} = \frac{200}{10000} = 0.02$

With Linear Interpolation

Choose $\lambda_1 = 0.6, \lambda_2 = 0.3, \lambda_3 = 0.1$:

$$\hat{P}(\text{cat}|\text{the quick}) = 0.6 \times 0 + 0.3 \times 0.01 + 0.1 \times 0.02 = 0 + 0.003 + 0.002 = 0.005$$

Even though the trigram was zero, we still get a reasonable probability!

Example

- Corpus counts:
- count("quick brown jumps") = 5
- count("quick brown") = 10
- count("brown jumps") = 8
- count("brown") = 50
- count("jumps") = 100
- Total words N = 10,000

Check trigram

- count("quick brown jumps") = 5 > 0, so:

$$S(\text{jumps} \mid \text{quick brown}) = \frac{5}{10} = 0.5$$

Check trigram

- Calculate S(runs|quick brown)
- count("quick brown runs") = 0 (doesn't exist)
- So we back off to bigram: 0.4·S(runs|brown)
- Check bigram:
- count("brown runs") = 3
- count("brown") = 50

Check trigram

- S(runs|brown)= 3 / 50 =0.06
- S(runs|quick brown)=0.4×0.06=0.024

Advantages

- Extremely fast - No normalization required
- Memory efficient - Only need counts, not probabilities
- Scalable - Works with billions of n-grams
- Simple implementation - Just if-else statements